Architecture

How it works and why it's built this way.

For: Everyone

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

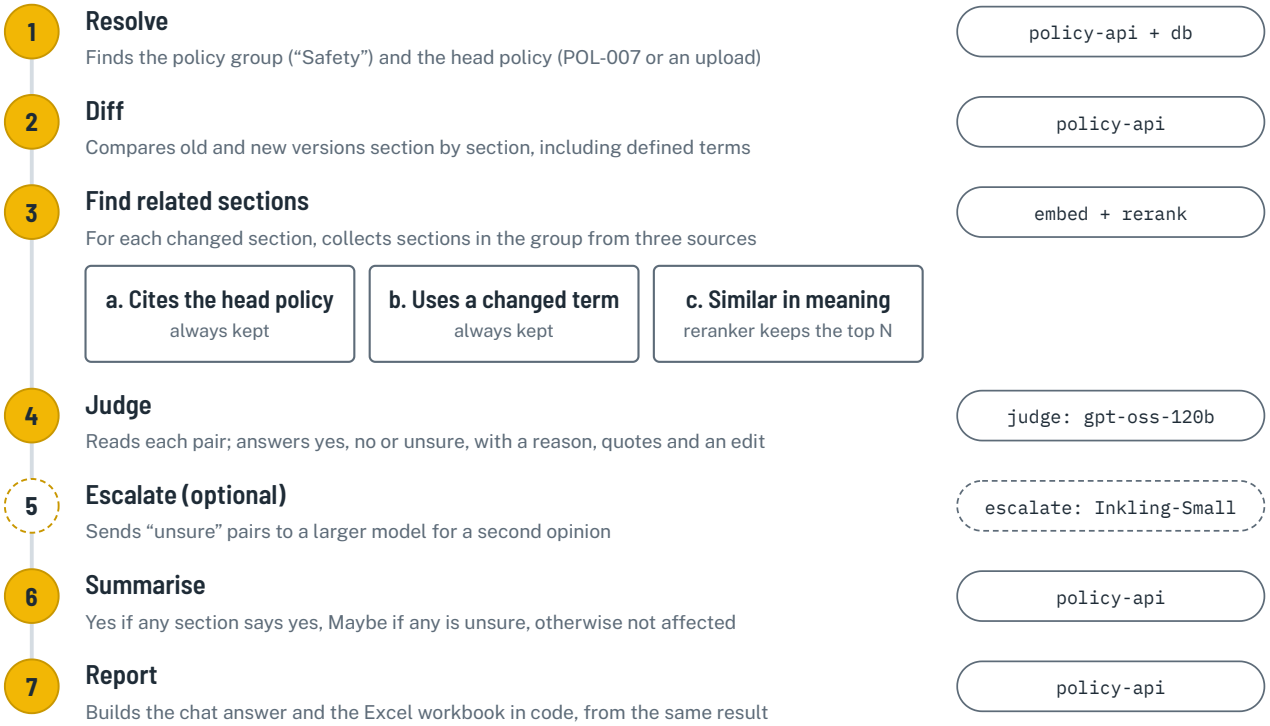
The job

When a head policy changes, find every policy in a chosen group that now conflicts with it, repeats its old wording, points to a section that moved, or uses a defined term whose meaning changed. Missing an affected policy is the expensive mistake, so the design favours **recall** (finding everything) over precision (flagging nothing extra), and it puts a person in the loop for anything uncertain.

The core rule

The chat model never decides impact itself. It works out what the person is asking, calls the pipeline through a tool, and passes on the result. The pipeline makes every decision, and code builds every number, table and spreadsheet. The chat answer and the Excel file therefore always agree, and every decision can be traced in the audit trail.

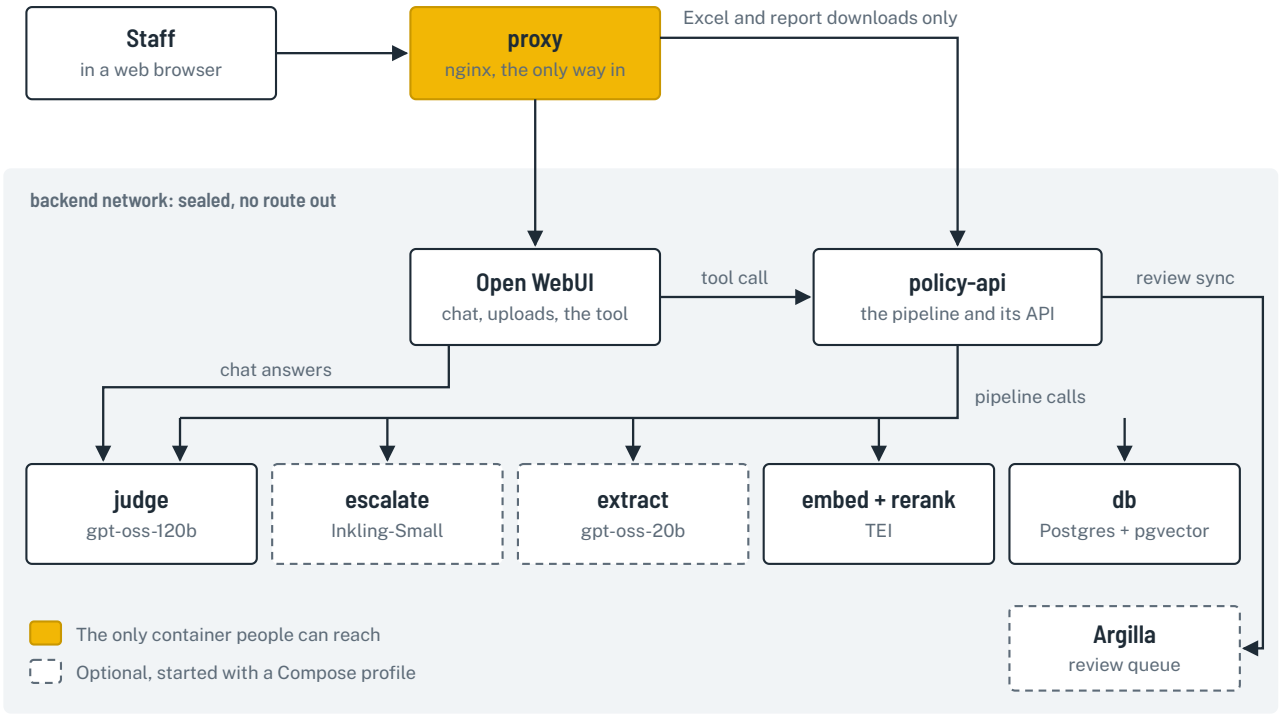
How one question is answered



How one question is answered. Code makes every decision; the judge rules on one pair at a time.

Steps 3a and 3b are never cut by the reranker, because explicit links are where most real impact is. Step 3c catches policies that overlap in meaning without citing the head policy.

Components



What runs where. People reach only the proxy; every service that holds policy text is on a sealed network.

Component	What it does	Network
`judge` (vLLM, gpt-oss-120b)	Judges pairs; also the chat model in Open WebUI (tool calling on)	backend
`escalate` (vLLM, Inkling-Small)	Optional second opinion on "unsure" pairs	backend
`extract` (vLLM, gpt-oss-20b)	Optional: finds references the regex misses during ingest	backend
`embed`, `rerank` (TEI)	Embeddings for similarity search; reranker to cut candidates	backend
`db` (Postgres + pgvector)	Policies, versions, clauses, reference graph, runs, audit trail	backend
`policy-api` (FastAPI)	The pipeline and its HTTP API	backend
`open-webui`	Chat interface, file upload, runs the tool	backend
`proxy` (nginx)	The only thing people reach: Open WebUI, plus Excel downloads	backend + frontend

The `backend` network is `internal: true`, so it has no route out of the machine. Only the proxy also joins the user-facing network, and it holds no data. See security.md.

Why these choices

Choice	Why	Instead of
A fixed pipeline with a chat front end	The steps never change. A pipeline is deterministic, testable and auditable	An agent that plans its own steps (more tokens, results vary between runs, harder to audit)
gpt-oss-120b as judge	US-developed, Apache 2.0, about 5B active parameters, fits one 80 GB GPU, high throughput, structured output	Inkling for everything (needs 4–8 high-end GPUs per replica)
Inkling-Small only for "unsure"	Larger-model judgement where it matters, at a fraction of the cost	Running every pair through the large model
Reranker before judging	Cuts hundreds of similar clauses to about 20 per change; the biggest compute saving	Judging every similar clause
JSON-schema guided decoding	Every verdict is valid and machine-readable	Asking for JSON in the prompt and hoping
Postgres for the graph	Recursive queries cover policy hierarchies; one fewer system to run	A separate graph database
Excel built with openpyxl	Exact, repeatable, safe from formula injection	Letting the model write the spreadsheet
Job queue in Postgres (`FOR UPDATE SKIP LOCKED`, leases)	Runs survive restarts with no new service to run or secure; Postgres is already here	Temporal or Dagster: another stateful system to operate inside the contained network, for a queue of a few hundred jobs a day

Data model (db/init.sql)

- `policies`, `policy_versions`, `clauses`: every version is kept. `current_clauses` is a view of each policy's latest version.
- `refs`: the reference graph (which clause cites which policy and section), for latest versions.
- `subsets`: which policies belong to which group. A policy can be in several groups.
- `uploads`, `runs`, `changes`: each question and what changed.
- `judgements`: one row per model decision, with model name, the exact prompt and its hash, verdict, and the reviewer's decision. This is the audit trail and the training data.
- `jobs`: the work queue (see operations.md). `schema_migrations` records which files in `policy_api/app/migrations/` have been applied; `db/init.sql` is the starting point for a new database.

Clause parsing rules

A new clause starts at a line that begins with a section number:

- a Markdown heading with a number (`## 4 Lockout devices`)
- any multi-level number (`4.2 Devices are inspected…`, `4.2.1) …`)
- a single number followed by a short title (`4 Lockout devices`)

`1. Do this.` inside a clause stays part of that clause. Text before the first number is clause `0`. Policy IDs are found with `POLICY_ID_PATTERN` (default `[A-Z]{2,6}-\d{1,5}`), in forms like `POL-007 §4.2`, `§4.2 of POL-007`, `POL-007 section 4` and `POL-007` alone (whole policy).

Roadmap

Built since v0.1: the job queue (operations.md), the review queue and LoRA fine-tuning (review-and-training.md), and reports, comparisons and charts (how-to.md).

1. **OCR** for scanned PDFs, and reading Word's automatic numbering (see known limits in how-to.md).
2. **Kubernetes** manifests when the models need more than one node.